

Rapport d'Évaluation - Projet Planning Poker en Python

Réalisé par : **DJERDI Ilyas Kaci & SEDDIK Oussama**

Résumé Exécutif

Le projet "Planning Poker GUI" est une application complète de planification agile développée en Python avec interface graphique Tkinter. L'application implémente les fonctionnalités essentielles du Planning Poker avec une architecture modulaire et une documentation professionnelle.

Explication du Choix du Langage de Programmation

Pourquoi Python ?

J'ai choisi Python comme langage de programmation principal pour ce projet pour plusieurs raisons stratégiques et pratiques :

1. Productivité et Rapidité de Développement

Syntaxe lisible et expressive : Python permet de développer des fonctionnalités complexes avec moins de code

Prototypage rapide : Idéal pour les projets académiques avec des délais serrés

Développement itératif : Permet d'ajouter des fonctionnalités progressivement

2. Écosystème Riche pour les Interfaces Graphiques

Tkinter intégré : Disponible par défaut avec Python, pas d'installation supplémentaire nécessaire

Communauté active : Nombreux tutoriels et ressources pour Tkinter

Multiplateforme : L'application fonctionne sur Windows, macOS et Linux sans modification

3. Bibliothèques Standard Puissantes

Exemples des modules Python utilisés :

```
import json
```

```
import tkinter as tk
```

```
import statistics
```

```
import os
```

Ces bibliothèques standard ont réduit la nécessité de dépendances externes.

4. Idéal pour le Traitement des Données

Manipulation facile des structures de données (listes, dictionnaires)

Sérialisation JSON native pour la sauvegarde des sessions

Calculs statistiques simplifiés pour les modes de vote

5. Documentation et Outillage

Doxygen supporte Python : Génération automatique de documentation

Outils d'analyse statique : flake8, pylint, black (potentiellement utilisables)

Tests unitaires intégrés : module unittest dans la bibliothèque standard

6. Adapté aux Projets Académiques

Courbe d'apprentissage douce : Accessible même avec une expérience limitée

Focus sur la logique métier : Moins de temps passé sur la syntaxe complexe

Débogage facilité : Messages d'erreur clairs et interpréteur interactif

7. Intégration Continue Simplifiée

GitHub Actions a un excellent support pour Python

Installation simple des dépendances

Exécution sur plusieurs versions de Python facile à configurer

Alternatives Considérées et Rejetées

Langage	Pourquoi non choisi
---------	---------------------

Java	Plus verbeux, configuration complexe pour une petite application
------	--

JavaScript/Electron	Dépendances lourdes, moins adapté au traitement de données
---------------------	--

C#	Limitée à l'écosystème Windows pour les applications desktop
----	--

C++	Développement plus lent, gestion manuelle de la mémoire
-----	---

Limitations de Python pour ce Projet

Bien que Python soit un excellent choix, j'ai rencontré certaines limitations :

Performance de Tkinter : Interface moins moderne que d'autres frameworks

Packaging : Création d'exécutables standalone plus complexe (PyInstaller, cx_Freeze)

Typage dynamique : Moins de vérifications à la compilation

Analyse Technique

Points Forts

Architecture Modulaire : Séparation claire des responsabilités avec 5 modules distincts

Interface Utilisateur Complète : Interface Tkinter intuitive avec gestion des votes et des joueurs

Gestion des Données Robustes : Sauvegarde/chargement JSON avec mécanisme de récupération

Documentation Professionnelle : Intégration Doxygen automatisée avec génération CI/CD

Flexibilité des Modes de Vote : 5 algorithmes de résolution différents.

Utilisation de l'IA Générative

Dans la section Models : J'ai utilisé l'IA générative pour m'aider à structurer les classes principales et à comprendre les meilleures pratiques de conception. L'IA a été particulièrement utile pour :

- L'organisation des méthodes et attributs selon le principe de responsabilité unique
- La réflexion sur les patterns de conception adaptés à une application Tkinter

L'IA a servi d'assistant pour accélérer le développement et les tâches peu cérébrales comme la détection des erreurs de syntaxes.

Structure du Code

Projet-Gestion-de-projet-agile-Planning-Poker-/

```
|— main.py      # Point d'entrée (conforme aux standards)
|— models.py    # Classes 3etier (Task, PlanningPokerGUI)
|— utils.py     # Fonctions utilitaires et calculs
|— io_helpers.py # Gestion I/O (JSON, modales)
|— config.py    # Configuration centralisée
|— .github/workflows/ # Pipeline CI/CD moderne
└— docs/       # Documentation générée
```

Évaluation de la Qualité du Code

Points Positifs

- Documentation Automatisée : Intégration Doxygen complète avec GitHub Actions
- Gestion d'Erreurs : Présence de fonctions comme `safe_save()` et `load_backup()`
- Séparation des Préoccupations : Architecture en couches claire
- Maintenabilité : Configuration externalisée dans `config.py`
- Versionnage : Présence de `projet_ppgp.py` pour l'historique

Points à Améliorer

- Absence de Tests Unitaires

Intégration Continue (CI) : J'ai implémenté un pipeline GitHub Actions pour la génération automatique de documentation, mais je n'ai pas réalisé de tests unitaires. Pour une véritable intégration continue complète, il manque :

Tests Automatiques : Pas de suite de tests pour valider les fonctionnalités

Les tests ont été fait à la main pour vérifier le fonctionnement de l'application à chaque fois ce qui mis énormément de temps.

Conclusion

Le choix de Python s'est avéré judicieux pour ce projet de Planning Poker. Il a permis :

- Un développement rapide des fonctionnalités principales
- Une architecture modulaire et maintenable
- Une documentation automatique de qualité
- Une intégration continue simplifiée

Leçon apprise : Bien que Python ait accéléré le développement, l'absence de tests unitaires est une lacune importante. Dans un projet futur, j'intégrerais les tests dès le début du développement.

Le projet démontre une bonne maîtrise de Python et de ses écosystèmes pour le développement d'applications desktop avec interface graphique. L'utilisation judicieuse des bibliothèques standard et de Tkinter montre une compréhension pratique des forces de Python pour ce type d'application.